

Temporal CLI Cheatsheet

Server

```
# Start local dev server (in-memory, no Docker)
temporal server start-dev

# Start with persistent SQLite storage
temporal server start-dev --db-filename clusterdata.db

# Custom ports
temporal server start-dev --port 7233 --ui-port 8233

# Custom namespace
temporal server start-dev --namespace my-namespace

# Headless (no Web UI)
temporal server start-dev --headless
```

Web UI: <http://localhost:8233> **gRPC:** localhost:7233

Workflow — Start

```
# Start (non-blocking - returns workflow ID)
temporal workflow start \
  --type WorkflowTypeName \
  --task-queue queue-name \
  --workflow-id my-wf-01 \
  --input '"string input"'

# Start with JSON object input
temporal workflow start \
  --type WorkflowTypeName \
  --task-queue queue-name \
  --workflow-id my-wf-01 \
  --input '{"key": "value"}'

# Start with no input
temporal workflow start \
  --type WorkflowTypeName \
  --task-queue queue-name \
  --workflow-id my-wf-01 \
  --input '{} '

# Start with execution timeout
temporal workflow start \
```

```
--type WorkflowTypeName \  
--task-queue queue-name \  
--workflow-id my-wf-01 \  
--execution-timeout 60s \  
--input '{}'  
  
# Start with custom namespace  
temporal workflow start \  
  --type WorkflowTypeName \  
  --task-queue queue-name \  
  --workflow-id my-wf-01 \  
  --namespace my-namespace \  
  --input '{}'
```

Workflow — Execute

```
# Execute and wait for result (blocking)  
temporal workflow execute \  
  --type WorkflowTypeName \  
  --task-queue queue-name \  
  --workflow-id my-wf-01 \  
  --input '{}'
```

Workflow — List

```
# List recent workflows  
temporal workflow list  
  
# List all open (running) workflows  
temporal workflow list --query 'ExecutionStatus = "Running"  
  
# List by workflow type  
temporal workflow list --query 'WorkflowType = "MyWorkflow"  
  
# List by task queue  
temporal workflow list --query 'TaskQueue = "my-queue"  
  
# List with custom namespace  
temporal workflow list --namespace my-namespace
```

Workflow — Show (Event History)

```
# Show event history (brief)
temporal workflow show --workflow-id my-wf-01

# Show with full payload details
temporal workflow show --workflow-id my-wf-01 --detailed

# Show with specific namespace
temporal workflow show --workflow-id my-wf-01 --namespace my-namespace
```

Workflow — Describe

```
# Describe workflow execution metadata
temporal workflow describe --workflow-id my-wf-01

# With run ID
temporal workflow describe --workflow-id my-wf-01 --run-id <run-id>
```

Workflow — Cancel

```
# Graceful cancel (workflow receives CancellationError, can clean up)
temporal workflow cancel --workflow-id my-wf-01

# With reason
temporal workflow cancel --workflow-id my-wf-01 --reason "cancelled by
admin"
```

Workflow — Terminate

```
# Immediate termination (no cleanup, workflow stops instantly)
temporal workflow terminate --workflow-id my-wf-01

# With reason (appears in event history)
temporal workflow terminate --workflow-id my-wf-01 --reason "stuck - force
stopping"
```

Workflow — Signal

```
# Send signal (no input)
temporal workflow signal \
```

```
--workflow-id my-wf-01 \  
--name signal-name  
  
# Send signal with string input  
temporal workflow signal \  
  --workflow-id my-wf-01 \  
  --name signal-name \  
  --input '"value"'  
  
# Send signal with JSON input  
temporal workflow signal \  
  --workflow-id my-wf-01 \  
  --name approve \  
  --input '{"approved": true}'  
  
# Send signal with integer input  
temporal workflow signal \  
  --workflow-id my-wf-01 \  
  --name queue_file \  
  --input '42'
```

Workflow — Query

```
# Query workflow state  
temporal workflow query \  
  --workflow-id my-wf-01 \  
  --type query-name  
  
# Query with input  
temporal workflow query \  
  --workflow-id my-wf-01 \  
  --type get_balance \  
  --input '{"account": "acc-01"}'
```

Workflow — Update

```
# Send update (blocks until update completes, returns result)  
temporal workflow update \  
  --workflow-id my-wf-01 \  
  --name update-name \  
  --input '{"a": 9, "b": 3, "op": "divide"}'  
  
# Update with update ID (for idempotency)  
temporal workflow update \  
  --workflow-id my-wf-01 \  
  --name run_calculator \  
  --update-id my-upd-01
```

```
--update-id calc-op-01 \  
--input '{"a": 9, "b": 3, "op": "divide"}'
```

Task Queue — Inspect

```
# Describe task queue (shows workers, pollers)  
temporal task-queue describe --task-queue queue-name  
  
# With namespace  
temporal task-queue describe --task-queue queue-name --namespace my-  
namespace  
  
# Show specific task queue type  
temporal task-queue describe --task-queue queue-name --task-queue-type  
workflow  
temporal task-queue describe --task-queue queue-name --task-queue-type  
activity
```

Namespace

```
# List all namespaces  
temporal operator namespace list  
  
# Describe a namespace  
temporal operator namespace describe --namespace my-namespace  
  
# Create a namespace  
temporal operator namespace create my-namespace  
  
# Update retention period  
temporal operator namespace update --namespace my-namespace --retention 30d
```

Batch Operations

```
# Batch terminate by query  
temporal workflow terminate \  
  --query 'WorkflowType = "MyWorkflow" AND ExecutionStatus = "Running"' \  
  --reason "batch cleanup"  
  
# Batch cancel by query  
temporal workflow cancel \  
  --query 'TaskQueue = "old-queue" AND ExecutionStatus = "Running"'
```

Common Flags (all commands)

Flag	Default	Description
<code>--address</code>	<code>localhost:7233</code>	Temporal gRPC endpoint
<code>--namespace</code>	<code>default</code>	Temporal namespace
<code>--tls</code>	<code>false</code>	Enable TLS
<code>--tls-cert-path</code>	—	Client cert path
<code>--tls-key-path</code>	—	Client key path
<code>--codec-endpoint</code>	—	Custom data converter endpoint

Quick Reference: cancel vs terminate

Command	Workflow receives	Use when
<code>cancel</code>	<code>CancellationError</code>	Workflow has cleanup logic
<code>terminate</code>	Nothing (immediate)	Workflow is stuck / must stop now